Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Институт информатики и вычислительной техники

09.03.01 "Информатика и вычислительная техника"
профиль "Программное обеспечение средств
вычислительной техники и автоматизированных систем"

Кафедра прикладной математики и кибернетики

Курсовая работа по дисциплине

Сетевое программирование

Разработка сетевого приложения. TCP клиент, PTHREAD.

Выполнил:

студент гр.ИП-111 _____/Гердележов Д.Д./
ФИО студента

«11» Мая 2024 г.

Проверил

_____/Ревун А.Л./
ФИО преподавателя

«13» Мая 2024 г. Оценка_____

Новосибирск 2024 г.

Оглавление

1. Постановка задачи: 3
2. Описание протокола 4
2.1. Основные характеристики: 4
2.2. Принцип работы: 5
3. Программная реализация 6
Описание программы 6
3.1. Клиент 6
3.2. Сервер 7
4. Пример работы программы 8
5. Листинг 9
5.1. Клиент 9
5.2. Сервер 11
5.3. constants.h 14
6. Список использованных источников 15

1. Постановка задачи:

Разработать консольное сетевое приложение на языке программирования C/C+ в OS Linux «Чат». Мультипоточная реализация сервера, на базе протокола TCP; PTHREAD.

2. Описание протокола

Протокол TCP (Transmission Control Protocol) — один из основных протоколов передачи данных в сети Интернет. Его основная цель — обеспечить надежную передачу данных между устройствами. Функционирует на транспортном уровне модели OSI. Разработан Винтоном Серфом и Робертом Каном в 1970-х годах.

2.1. Основные характеристики:

1. **Надежность:** TCP гарантирует доставку пакетов данных в правильном порядке и без ошибок. Он достигает этого с помощью механизмов, таких как подтверждение, повторная передача и упорядочивание.
2. **Ориентированность на соединение:** перед началом передачи данных TCP устанавливает соединение между отправителем и получателем через трехступенчатое рукопожатие. Это обеспечивает согласование обеих сторон по параметрам коммуникации и позволяет упорядочить обмен данными.
3. **Управление потоком:** TCP использует механизмы управления потоком для управления скоростью передачи данных между устройствами. С помощью таких техник, как оконный механизм, он предотвращает перегрузку получателя большим объемом данных, чем он способен обработать, тем самым оптимизируя производительность сети.
4. **Управление перегрузкой:** TCP динамически регулирует скорость передачи данных в зависимости от уровня сетевой перегрузки, чтобы предотвратить потерю пакетов и поддерживать общую стабильность сети.
5. **Двунаправленная передача:** TCP поддерживает одновременную, двунаправленную передачу данных, позволяя обеим сторонам отправлять и получать данные независимо. Эта возможность повышает эффективность сети, обеспечивая одновременное взаимодействие.

2.2. Принцип работы:

1. **Установка соединения:** TCP начинает коммуникацию с трехступенчатым рукопожатием, где отправитель отправляет пакет SYN, получатель отвечает пакетом SYN-ACK, и, наконец, отправитель подтверждает получение пакета ACK.
2. **Передача данных:** после установления соединения TCP разбивает данные на управляемые блоки, известные как сегменты, и передает их по сети. Каждый сегмент содержит номера последовательности и контрольные суммы для обнаружения ошибок и упорядочивания.
3. **Подтверждение:** после получения сегментов данных получатель отправляет пакеты подтверждения (ACK) обратно отправителю, подтверждая успешное получение. Если отправитель не получает подтверждения, он повторно отправляет сегмент данных.
4. **Управление потоком и перегрузкой:** TCP использует алгоритмы для управления потоком, позволяя отправителю регулировать скорость передачи данных в зависимости от возможностей буфера получателя.
5. **Завершение соединения:** когда обмен данными завершается, TCP завершает соединение через четырехступенчатое рукопожатие. Отправитель инициирует завершение, отправляя пакет FIN, за которым следует ACK от получателя. Затем получатель отправляет свой собственный пакет FIN, и отправитель отвечает окончательным ACK, закрывая соединение.

3. Программная реализация

Конечный программный продукт состоит из двух частей: сервер и клиент, запуск производится с помощью одноименных файлов: client, server.

Описание программы

3.1. Клиент

1. **sendingMsgs -** функция выполняется в отдельном потоке и отвечает за отправку сообщений на сервер.
    - Получает дескриптор сокета клиента (**clientSocketHandle**) в качестве аргумента.
    - В цикле считывает сообщения с клавиатуры с помощью **fgets**.
    - Если введенное сообщение превышает максимальную длину (**MSG_SIZE**), уведомляет об этом и обрезает сообщение до допустимой длины.
    - Отправляет сообщение на сервер с помощью **send**.
2. **connectServer** - твечает за подключение к серверу.
    - Создает сокет клиента (**clientSocket**) с помощью **socket**.
    - Настраивает структуру **sockaddr_in** для подключения к серверу по указанному хосту и порту.
    - Выполняет подключение к серверу с помощью **connect**.
    - Отправляет никнейм клиента на сервер с помощью **send**.
3. **Функция main**:
    - Проверяет правильность переданных аргументов командной строки (хост, порт, никнейм).
    - Создает поток **sendingMsgs** для отправки сообщений.
    - В главном потоке принимает сообщения от сервера с помощью **recv**.
    - При получении сообщения выводит его на экран.
    - В случае ошибки при получении сообщения выводит уведомление об ошибке.

- В случае завершения работы сервера закрывает сокет и завершает программу.

3.2. Сервер

1. **sendToAll -** отправляет сообщение **msg** всем подключенным клиентам. Она блокирует мьютекс, чтобы предотвратить одновременный доступ к вектору клиентов, затем перебирает всех клиентов в векторе и отправляет каждому сообщение. Если отправка не удалась, функция выведет сообщение об ошибке в стандартный поток ошибок.
2. **removeClient -** удаляет клиента из вектора клиентов по его идентификатору **id**. Она также блокирует мьютекс для безопасного доступа к вектору клиентов.
3. **handleClient -** обрабатывает соединение с каждым клиентом в отдельном потоке. Она начинает с извлечения сокета клиента из параметра **clientSocketHandlePtr**, затем принимает его никнейм, создает новую структуру клиента и добавляет его в вектор клиентов. Затем она входит в бесконечный цикл приема сообщений от клиента, отправки их всем остальным клиентам и удаления клиента в случае отключения.
4. **main -** главная функция программы. Она начинается с проверки количества аргументов командной строки. Затем она создает сокет сервера, устанавливает его параметры и запускает прослушивание. Затем она входит в бесконечный цикл приема новых клиентов, создавая для каждого нового клиента отдельный поток выполнения с функцией **handleClient**.

4. Пример работы программы

Тестирование будет проводиться следующим образом, сначала 2 пользователя обоняются сообщениями, затем к ним подлечится, напишет сообщение и отключится пользователь номер 3.

Сервер был запущен на удаленной машине:

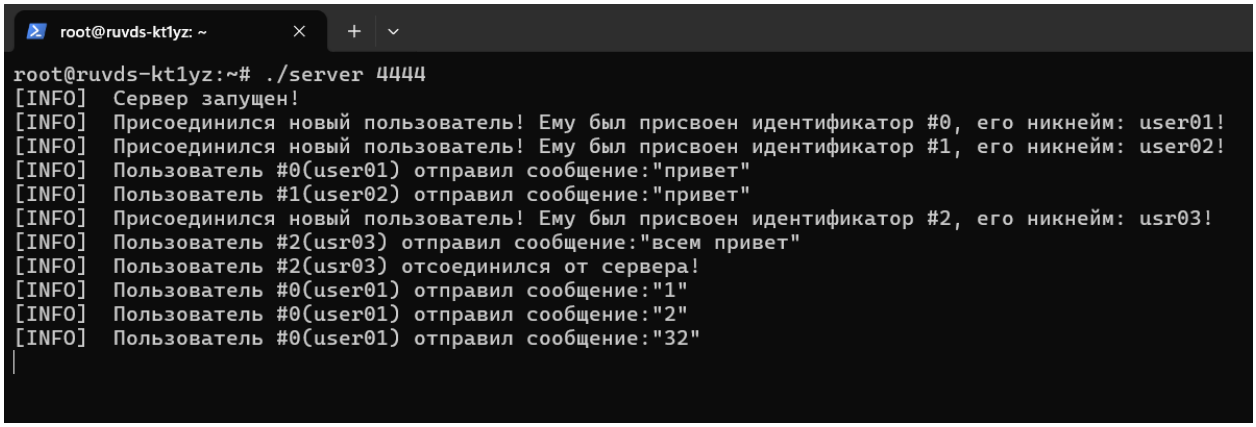


Рис. 1 – Результат работы серверной части.



Рис. 2 – Команда для подключения usr03 к чату.



Рис. 3 – Пример использования чата со стороны клиентов (слева на право user01, user02, user03).

Как видно из рисунка 3 при последовательной отправке сообщений ошибок не возникает.

5. Листинг

5.1. Клиент

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <string.h>
#include <iostream>
#include <pthread.h>
#include <sys/socket.h>
#include <unistd.h>

#include "constants.h"


void* sendingMsgs(void* arg) {
    int clientSocketHandle = *((int*)arg);
    char msg[MSG_SIZE + 1];
    char* pos;
    char c;

    while (true) {
        fgets(msg, sizeof(msg), stdin);

        if ((pos = strchr(msg, '\n')) != nullptr) {
            *pos = '\0';
        } else {
            fprintf(stdout,
                    "[Notification]\tCooобщение превысило допустимую длину. "
                    "Сообщение обрезано до %d символов\n",
                    MSG_SIZE);
            msg[sizeof(msg) - 1] = '\0';
            while ((c = getchar()) != EOF && c != '\n')
                ;
        }
        if (send(clientSocketHandle, msg, sizeof(msg), 0) < 0)
            fprintf(stderr,
                    "[Error]\tНе удалось отправить сообщение.\nПроверьте "
                    "подключение и повторите попытку\n");
    }
    return nullptr;
}

int connectServer(const char* host, const char* port, const char* name) {
    int clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (clientSocket == -1) {
        fprintf(stderr, "[Error]\tНе удалось создать clientSocket\n");
        return -1;
    }
```

```
    struct sockaddr_in clientAddr;
    clientAddr.sin_family = AF_INET;
    clientAddr.sin_port = htons(strtol(port, nullptr, 0));
    if (inet_aton(host, &clientAddr.sin_addr) == 0) {
        fprintf(stderr, "[Error]\tНекорректный IP\n");
        return -1;
    }

    if (connect(clientSocket, (struct sockaddr*)&clientAddr,
                sizeof(sockaddr_in)) < 0) {
        fprintf(stderr,
                "[Error]\tНе удалось подключиться к серверу.\nПроверьте "
                "подключение и доступность сервера\n");
        return -1;
    }

    if (send(clientSocket, name, strlen(name), 0) < 0) {
        fprintf(stderr,
                "[Error]\tНе удалось отправить никнейм на сервер.\nПроверьте "
                "подключение и повторите попытку\n");
        return -1;
    }

    return clientSocket;
}

int main(int argc, char* argv[]) {
    system("clear");
    if (argc != 4) {
        fprintf(stderr, "./client [host] [port] [nickname]");
        return 1;
    }

    char name[MAX_NICKNAME_SIZE + 1]{'\0'};
    if (strlen(argv[3]) > MAX_NICKNAME_SIZE) {
        argv[3][MAX_NICKNAME_SIZE] = '\0';
        fprintf(stdout,
                "[Notification]\tДлина никнейка превышает допустимое "
                "значение(%d). Никнейм обрезан до \"%s\"\n",
                MAX_NICKNAME_SIZE, argv[3]);
    }

    strcpy(name, argv[3]);
    int clientSocket;
    if ((clientSocket = connectServer(argv[1], argv[2], name)) == -1) {
        return 3;
    }

    fprintf(stdout,
            "\n*************************** CHAT "
```

```
            "**************************\n");

    pthread_t thread;
    if (pthread_create(&thread, NULL, sendingMsgs, &clientSocket)) {
        fprintf(stderr, "Error creating thread\n");
        return 1;
    }

    int bytesRecv;
    char msg[BUFFER_SIZE];
    while ((bytesRecv = recv(clientSocket, msg, sizeof(msg), 0))) {
        if (bytesRecv < 0) {
            fprintf(stderr,
                    "[Error]\tОшибка при получении сообщения от сервера\n");
        } else {
            fprintf(stdout, "%s\n", msg);
        }
        memset(msg, '\0', sizeof(msg));
    }
    std::cerr << "[Error]\tСервер завершил свою работу\n" << std::endl;
    close(clientSocket);

    return 0;
}
```

5.2. Сервер

```
#include <netinet/in.h>
#include <pthread.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#include <cstdlib>
#include <vector>

#include "constants.h"

struct client {
    int socketHandle;
    int id;
    char nickname[MAX_NICKNAME_SIZE + 1]{'\0'};
};

std::vector<client> clients;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void sendToAll(char msg[BUFFER_SIZE]) {
    pthread_mutex_lock(&mutex);
    for (unsigned int i = 0; i < clients.size(); i++) {
```

```
        if (send(clients[i].socketHandle, msg, BUFFER_SIZE, 0) < 0)
            fprintf(
                stderr,
                "[ERROR]\tНе удалось отправить сообщение пользователю \"%s\"\n",
                clients[i].nickname);
    }
    pthread_mutex_unlock(&mutex);
}

void removeClient(int id) {
    int index = -1;

    pthread_mutex_lock(&mutex);
    for (unsigned int i = 0; i < clients.size(); i++) {
        if (clients[i].id == id) index = (int)i;
    }
    if (index != -1) {
        clients.erase(clients.begin() + index);
    } else {
        fprintf(stderr, "[ERROR]\tНе удалось удалить клиента из вектора\n");
    }
    pthread_mutex_unlock(&mutex);
}

void* handleClient(void* clientSocketHandlePtr) {
    int clientSocketHandle = *(int*)clientSocketHandlePtr;
    free(clientSocketHandlePtr);

    char name[MAX_NICKNAME_SIZE + 1];
    char msg[MSG_SIZE + 1];
    char buffer[BUFFER_SIZE];

    struct client* newClient = new client();
    newClient->id = clients.size();
    newClient->socketHandle = clientSocketHandle;

    memset(buffer, 0, sizeof(buffer));
    int bytesRecv;

    if (recv(clientSocketHandle, name, sizeof(name), 0) < 0) {
        delete newClient;
        return nullptr;
    } else {
        strcpy(newClient->nickname, name);
        fprintf(stdout,
                "[INFO]\tПрисоединился новый пользователь! Ему был присвоен "
                "идентификатор #%d, его никнейм: %s!\n",
                newClient->id, newClient->nickname);
        send(newClient->socketHandle, "[SERVER]\tДобро пожаловать в чат!",
             sizeof("[SERVER]\tДобро пожаловать в чат!"), 0);
```

```
        buffer[0] = '\0';
        strcat(buffer, "[SERVER]\t");
        strcat(buffer, newClient->nickname);
        strcat(buffer, "Присоединился к чату");
        sendToAll(buffer);
        pthread_mutex_lock(&mutex);
        clients.push_back(*newClient);
        pthread_mutex_unlock(&mutex);
    }

    while (true) {
        buffer[0] = '\0';
        bytesRecv = recv(clientSocketHandle, msg, sizeof(msg), 0);

        if (bytesRecv == 0) {
            fprintf(stdout,
                    "[INFO]\tПользователь #%d(%s) отсоединился от сервера!\n",
                    newClient->id, newClient->nickname);
            buffer[0] = '\0';
            strcat(buffer, "[SERVER]\t");
            strcat(buffer, newClient->nickname);
            strcat(buffer, " покинул чат...");
            sendToAll(buffer);

            close(clientSocketHandle);
            removeClient(newClient->id);
            break;
        } else if (bytesRecv < 0) {
            fprintf(stderr,
                    "[ERROR]\tОшибка при получении сообщения от клиента с "
                    "идентификатором #%d!\n",
                    newClient->id);
        } else {
            strcat(buffer, newClient->nickname);
            strcat(buffer, ">  ");
            strcat(buffer, msg);
            fprintf(stdout,
                    "[INFO]\tПользователь #%d(%s) отправил сообщение:\"%s\"\n",
                    newClient->id, newClient->nickname, msg);
            sendToAll(buffer);
        }
    }
    return nullptr;
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        fprintf(stderr, "./server [port]\nНапример, \"./server 2007\"\n");
        return 1;
    }
    char* port = argv[1];
```

```
    int serverSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (serverSocket == -1) {
        fprintf(stderr, "[Error]\tНе удалось создать serverSocket\n");
        return -1;
    }

    struct sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(strtol(port, nullptr, 0));

    if (bind(serverSocket, (struct sockaddr*)&serverAddr,
             sizeof(struct sockaddr_in)) < 0) {
        fprintf(stderr, "[Error]\tНе удалось присвоить сокету имя\n");
        return 1;
    }

    listen(serverSocket, SOMAXCONN);
    fprintf(stderr, "[INFO]\tСервер запущен!\n");

    while (true) {
        int* clientSocketHandlePtr = (int*)malloc(sizeof(int));
        *clientSocketHandlePtr = accept(serverSocket, nullptr, nullptr);
        if (*clientSocketHandlePtr < 0) {
            fprintf(stderr,
                    "[ERROR]\tОшибка при установлении соединения с клиентом\n");
            free(clientSocketHandlePtr);
        } else {
            pthread_t thread;
            if (pthread_create(&thread, NULL, handleClient,
                               (void*)clientSocketHandlePtr) != 0) {
                fprintf(stderr,
                        "[ERROR]\tОшибка при создании потока для клиента\n");
                free(clientSocketHandlePtr);
            } else {
                pthread_detach(thread);
            }
        }
    }

    return 0;
}
```

5.3. constants.h

```
#define BUFFER_SIZE        500
#define MAX_NICKNAME_SIZE  30
#define MSG_SIZE           BUFFER_SIZE-MAX_NICKNAME_SIZE-4-1
```

6. Список использованных источников

1. MDN Web Docs *“WebSockets”* [электронный ресурс] // https://developer.mozilla.org/ru/docs/Web/API/WebSockets_API
2. IBM “Протоколы TCP/IP” [электронный ресурс] // https://www.ibm.com/docs/ru/aix/7.2?topic=protocol-tcpip-protocols
3. Олифер Виктор, Олифер Наталья *“Компьютерные сети. Принципы, технологии, протоколы: Юбилейное издание.”* — СПб.: Питер, 2020. — 1008 с.: ил. — (Серия «Учебник для вузов»). ISBN 978-5-4461-1426-9
4. Курс лекций Павского К.В. «Сетевое программирование».